

Status Report I

1) Group Information

Group Number: 1

Member List: Nikita Aleksii, Robby Votta, Yurda Yolcu, and Maria Fajardo.

2) Project Snapshot

Project title: AssemblyViz

Primary customer: Dr. Hammurabi Mendes

Core goal: A web-based HYMN and RISC-V simulator and code editor that builds on the functionality of the software used by the department, with an improved user experience to improve student learning with assembly code.

Core novelty: AssemblyViz is a web application that integrates functionality from existing software for the HYMN assembly language (<https://cburch.com/socs/hymn/index.html>) and adds a history of memory cells and a debugger. It also implements RISC-V (built on this software: <https://peterhigginson.co.uk/RISC/>) visualization within the same web application, so that students and professors can choose which instruction set architecture (ISA) to use.

3) Major Changes Since Proposal

Report any significant changes made after the proposal submission. These may include changes to **the project goal, scope (features added or removed), customer or stakeholder priorities, user requirements, technical architecture, timeline, or development process.**

You do **not** need to report **minor edits**, such as small UI adjustments or routine implementation progress. If no major changes were made, clearly state so and briefly justify why the original proposal remains appropriate.

For each change, include:

Date	Change	Motivation (Why?)
2/18	Originally, backend developers planned to work on the HYMN implementation together in one sprint, then on the RISC-V implementation in another. We decided we did not need two developers working on each assembly language at the same time, and instead, merged the two sprints so that implementation and testing occurred in tandem. This was a team-driven change as we wanted to be able to link the frontend to both halves of our backend (RISC-V and HYMN) instead of having to do it twice. This meant we spent sprint 2 implementing the two systems and sprint 3 testing.	As stated in class, it is not necessary to have one individual coding and another watching over their shoulder. We have a small team, so we want to finish both languages at the same time so we can link them to the front end as soon as possible. We keep the scope of our project this way and increase testing time on RISC-V
3/5	Our current software implements a Python backend for RISC-V and HYMN. We originally promised a React/TypeScript web app for sprint 1 to complete the frontend framework, including basic UX design choices and the site's navigation structure for assembler components. Instead, we focused primarily on our backend logic in both HYMN and RISC-V assembly languages. This was a team-driven change to ensure comprehensive testing of our HYMN and RISC-V logic, which is the core of our project. So instead of focusing heavily on the frontend in sprint 1, we put our efforts towards HYMN and RISC-V implementations.	Instead of designing our UI-shell first and then coding our backend logistics around that, we now have a strong foundation with a working backend, and can more flexibly adjust the frontend.
3/10	Parser changed to a two-pass system to compile regex patterns on HYMN: First pass: Does not encode instructions. Records every label definition (e.g., 'FLAG:') and maps it to the address it will occupy in memory Second Pass: Encode every instruction into an 8-bit “word”. Uses the symbol table built by the first pass to resolve label operands. Changing parser.py to a two-pass system was a technical-driven change. We needed to allocate memory for labels <i>before</i> encoding instructions, not <i>during encoding</i>. Luckily, this	1. JMP END 2. ADD 5 3. END: 4. HLT In this example, END is not defined until line 3, but we need its address on line 1.

	change did not affect our time-frame. Our developers were able to complete the implementation by sprint 2.	
--	--	--

4) Sprint-by-Sprint Summary

Original sprint goals:

Sprint 1: Foundation & Architecture (Proposal – Report 1)

Objective: Establish the core application infrastructure for AssemblyViz

Deliverables:

- Complete project setup
- Frontend framework with basic UI shell and navigation structure
- Data models for assembler components (instructions, registers, memory)
- Technical architecture documentation

Out of scope: No assembler functionality implemented yet

Sprint 2: HYMN Assembler Implementation (Report 1 – Report 2)

Objective: Build a fully functional HYMN assembler with visualization capabilities

Deliverables:

- HYMN instruction parser and assembler
- Memory visualization with execution history
- Register display and tracking
- Interactive debugger

Out of scope: RISC-V support

Sprint 3: RISC-V Assembler Implementation (Report 2 – Final)

Objective: Support RISC-V assembly language

Deliverables:

- RISC-V instruction parser and assembler (RV32I base instruction set)
- Memory visualization, register display, and debugger matching HYMN functionality
- Architecture selection toggle (switch between HYMN and RISC-V modes)
- Final documentation and testing

Out of scope: Advanced RISC-V extensions beyond base integer instructions

Sprint #: 1

Period: February 3 – February 18

Sprint goal: Objective: Establish the core application infrastructure for AssemblyViz and begin implementing RISC-V and HYMN assembly logic.

Backlog items selected:

1. Different design iterations for the webpage in Figma
2. Navigation bar and logo
3. HYMN parser and executor Python files
4. RISC-V parser and executor Python files

Backlog items completed:

1. Final web-page design in Figma
2. Navigation bar and logo
3. HYMN parser and executor py files (untested)
4. RISC-V parser and executor py files (untested)

Artifact (What was demonstrated to the customer?):

1. Final Design for approval
2. Backend logic progress for both HYMN and RISC-V

Sprint #: 2

Period: February 18 – March 20

Sprint goal: Objective: Build a fully functional HYMN and RISC-V assembler, tested and ready to be linked to our frontend. Complete basic HTML shell frontend with code editor, register visualization, and memory

Backlog items selected:

1. Code editor
2. Register Result Section
3. Memory Allocation Chart
4. Backend test cases for HYMN and RISC-V

Backlog items completed:

1. HYMN parser and executor Python files (still having problems with logic on virus code)
2. RISC-V parser and executor Python files (fully implemented)
3. Code editor (can type any character into the input space)
4. Register Result Section (links each line to a register with no real logic)
5. Memory Allocation Chart (will not work till backend is linked)

Artifact (What was demonstrated to the customer?):

3. Live demo of user registration and login
4. Demonstration of profile editing
5. Database table showing stored user information
6. Walkthrough of error handling (invalid login attempt)

5) Customer Validation Across Sprints

Summarize customer engagement:

- **How many meetings?**
 - We met with our primary customer, Dr. Hammurabi Mendes, twice to validate whether we're developing the right software. During the first meeting, we validated the program design; during the second, we validated that the program implemented our primary customer's needs.
- **Format (Zoom / in-person)?**
 - In-person
- **Include one specific customer quote or a summarized feedback insight.**
 - Dr. Hammurabi Mendes was satisfied with both the design and the functionality we were developing. The software provides a user-friendly, intuitive interface that reduces psychological bias against programming in assembly. Also, the software correctly executes assembly languages by following the instruction set architectures manuals. Dr. Hammurabi Mendes mentioned he would provide additional feedback as we integrate the frontend with the backend.

6) Use Cases

Name: A user executing RISC-V assembly code

Actors: User and system (script-based emulator)

Goal: To successfully execute RISC-V assembly code and output all changes to memory and registers

Basic Flow:

- 1) The user chooses RISC-V, opens a simulation file, and inputs the assembly file name.
 {Open simulation file}
- 2) The user runs the simulation by executing a command in the terminal.
 {To Execute code}
- 3) The system executes assembly code following the RISC-V assembly language instruction guidelines and writes output to memory and registers.
 {Executing code}
- 4) The system finishes code execution.
 {Finish execution}
- 5) The user views the output after the system executes the code.
 {View the output}
- 6) Terminate the system
 {Halt}

Alternative Flows:

Specific Alternative Flows:

1. **When the user writes buggy assembly code.** If the system executes assembly code and encounters an unknown instruction at {Executing code}, it halts execution and warns the user. Then, the system moves to {Halt}.
2. **When the user stops code execution early.** If the user stops code execution in the terminal at {Executing code}, they must either move to {Halt}.

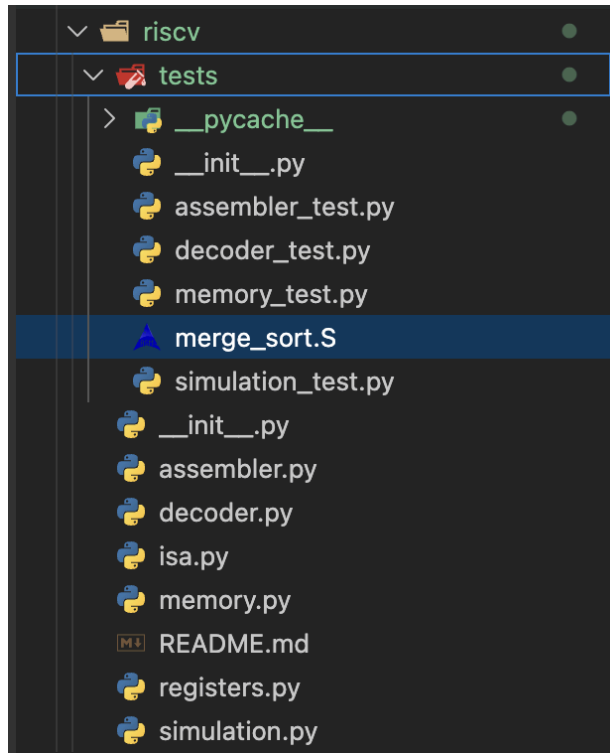
Bounded Alternative Flows:

1. **When a user's device cannot handle software execution.** If the user's device cannot continue execution between {To Execute code} and {Finish execution}, the software halts execution and goes to {Halt}.
2. **When the user attempts to execute the simulation without providing assembly code.** At any point between {To Execute code} and {Finish execution}, if the user doesn't provide assembly code but wants to run the simulation, don't output anything. Go to {Halt}.

Name: A user executing HYMN assembly code
Actors: User and system (script-based emulator)
Goal: To successfully execute HYMN assembly code and output all changes to memory and registers
Basic Flow: 1) The user chooses HYMN, opens a simulation file, and inputs the assembly file name. {Open simulation file} 2) The user runs the simulation by executing a command in the terminal. {To Execute code} 3) The system executes assembly code following the HYMN assembly language instruction guidelines and writes output to memory and registers. {Executing code} 4) The system finishes code execution. {Finish execution} 5) The user views the output after the system executes the code. {View the output} 6) Terminate the system {Halt}
Alternative Flows: Specific Alternative Flows: 1. When the user writes buggy assembly code. If the system executes assembly code and encounters an unknown instruction at {Executing code}, it halts execution and warns the user. Then, the system moves to {Halt}. 2. When the user stops code execution early. If the user stops code execution in the terminal at {Executing code}, the system halts the execution and moves to {Halt}. Bounded Alternative Flows: 1. When a user's device cannot handle software execution. If the user's device cannot continue execution between {To Execute code} and {Finish execution}, the software halts execution and goes to {Halt}. 2. When the user attempts to execute the simulation without providing assembly code. At any point between {To Execute code} and {Finish execution}, if the user doesn't provide assembly code but wants to run the simulation, don't output anything. Go to {Halt}.

7) Working Software

User Story: As a **student**, I want to **step through a RISC-V assembly program** one instruction at a time and see how registers and memory change at each step, so that I can **understand how the CPU executes my code**.



STEP 1

User Action: User writes a RISC-V assembly program (e.g., in this case, a merge sort implementation, merge_sort.S). The user places the program in the tests folder.

System Response: no response

Which system component is involved?

assembler.py will use a RISC-V assembly program to transform it into 32-bit instructions

Which acceptance test does this step support? None, since it is a setup.

```
backend > riscv > tests > simulation_test.py > main
1  from riscv.simulation import Simulation
2  from riscv.assembler import _labels
3
4  def main():
5      sim = Simulation(2048)
6
7      with open("../riscv/tests/merge_sort.S") as f:
8          src = f.read()
9
10     sim.load(src)
11
12     while not sim.snapshot()["halted"]:
13         sim.step()
14
15     arr_addr = _labels["array"]
16     arr_quantity = 7
17
18     for i in range(arr_quantity):
19         print(int(sim.memory.memory_read(arr_addr + i * 4), 2))
20
21 if __name__ == "__main__":
22     main()
```

STEP 2

User Action: User opens simulation_test.py and enters their file name on line 7. The user can also display a snapshot that includes memory, registers, the program counter, and the halted (program stopped) state.

System Response: no response

Which system component is involved? RISC-V instruction execution simulator

Which acceptance test does this step support? None, since it is a setup.

STEP 3

```
● nikitaaleksii@Mac backend % python3 -m riscv.tests.simulation_test
0
2
3
10
24
35
353
```

User Action: User executes the script by running the command above

System Response: The system executes the user-provided RISC-V program. In this case, the program outputs an array, which was sorted using merge sort. The array was initialized to [24, 10, 35, 2, 3, 0, 353], and the program outputs a sorted array [0, 2, 3, 10, 24, 35, 353].

Which system component is involved?

All modules were involved.

assembler.py - translates assembly language code into binary machine code

decoder.py - translates a RISC-V binary instruction into a set of signals and fields

isa.py - defines instruction set architecture

memory.py - byte-addressable memory which stores instructions and variables

registers.py - stores values in registers

simulation.py - combines all files above and serves as an execution engine.

Which acceptance test does this step support?

Given a student has written valid RISC-V assembly code, when the student simulates the terminal, then all instructions execute until the end of the instructions is reached, and the final values are printed to the terminal.

8) Testing Progress Across Sprints

SPRINT 1

1. Unit tests performed on the core function (verification).

During the first sprint, we primarily focused on designing the software and implementing a foundational backend component, such as a parser for both HYMN and RISC-V. For the parser, we didn't test it because backend module testing was outside the scope of this sprint.

2. Acceptance test performed with the user (validation).

We validated the design with our primary customer, Dr. Hammurabi Mendes, to ensure that our software interface is intuitive and easy to use. Also, we implicitly validated the implementation of RISC-V visualization, as Nikita is taking an independent study with our primary customer and has previously implemented RISC-V features.

3. Were the tests done manually or automated?

Primarily, the tests were conducted manually because verifying and validating the design cannot be automated; rather, human involvement is required to ensure a smooth user experience. As mentioned before, no tests were done to verify the parser.

SPRINT 2

1. Unit tests performed on the core function (verification).

The main goal of the sprint was to develop a fully functional backend for visualizing RISC-V and HYMN. Part of this goal was to develop unit tests to verify their correct implementations. Therefore, we included a unit test for each module that compared the computed value with the actual value. This testing allowed us to identify bugs in the code and verify the correct implementation.

2. Acceptance test performed with the user (validation).

Since Robby and Nikita had prior exposure to HYMN and RISC-V assembly programming, we validated that we were building the right product.

3. Were the tests done manually or automated?

Most tests were automated to streamline debugging because automated testing pinpoints the issue in code. Others were done manually: for example, to test the whole program, we had to manually edit a testbench to incorporate the desired assembly code.

9) Sprint Review

SPRINT 1

1. What delivered the most customer value?

The finalized web-page design in Figma delivered the most customer value in Sprint 1. Validating the UI design with Dr. Hammurabi Mendes early ensured the interface would be intuitive for students before any frontend implementation began. The initial HYMN and RISC-V parser and executor files also laid the groundwork for the core backend logic.

2. What features were rejected or revised?

The existing HYMN software supports editing memory cells by clicking on them. We decided to exclude this feature from AssemblyViz because, from our observations, most users do not use it.

3. Any emerging new requirements?

No.

4. Customer perception of quality so far?

Dr. Hammurabi Mendes is satisfied with the current software.

SPRINT 2

1. What delivered the most customer value?

The fully implemented RISC-V parser and executor delivered the most customer value in Sprint 2. This backend logic correctly executes assembly programs following the RISC-V

instruction set architecture, as demonstrated by successfully running a merge sort program. The HYMN backend is also largely implemented.

2. What features were rejected or revised?

We revisited Frontend HTML design that we were originally going to work through in the first spring

3. Any emerging new requirements?

No.

4. Customer perception of quality so far?

Dr. Hammurabi Mendes is satisfied with the current software.

10) Sprint Retrospective

Across the sprint, role distribution remained similar: Nikita and Robby focused on backend development, while Maria and Yurda worked on the frontend. Within the frontend work, Maria primarily handled design decisions, while Yurda focused on implementing the interface. Contribution was relatively balanced, with each member focusing on their assigned area.

Meeting frequency remained generally consistent, with weekly check-ins to track progress, as stated in the proposal. There were no significant changes in sprint length, as the team continued following the structure outlined in the proposal.

One notable change was in the tools used. Initially, the frontend was developed using plain HTML, CSS, and JavaScript to prototype the interface quickly. However, it was later decided that transitioning to React and TypeScript would be more appropriate for long-term development.

Overall, the sprint successfully delivered a working frontend interface and made progress on backend development, although early technical decisions could have improved future efficiency.

10 Risk and Mitigation

Summarize across sprints:

- **New risks discovered?**

The transition from plain HTML/CSS/JavaScript to React and TypeScript introduces integration risk, as the frontend will need to be re-linked to the backend in the new framework.

- **Old risks resolved?**

Not yet. Hosting and merging the frontend with the backend remain tremendous risks heading into Sprint 3.

- **Technical debt accumulating?**

Yes. The early decision to prototype in plain HTML rather than React means some frontend work will need to be redone, which adds technical debt going into the final sprint.